Active Learning of Temporal Specifications through Distinguishing Queries

Anonymous Authors

***Abstract*— Temporal specifications are essential for verification and monitoring, yet writing formulas that capture the intended behaviour is difficult. Active learning derives such specifications through interaction between a learner and a teacher, but remains underexplored for temporal logics. Existing approaches require the teacher to answer equivalence queries and to construct counterexamples, and they often rely on SAT-based learning procedures that restart the search after each new example. We present an active-learning framework for LTLf in which the teacher only labels traces and answers natural-language questions about the intended behaviour. The learner maintains an enumerated version space of candidate formulas, generates traces that distinguish candidates, and uses the teacher's hints to rank candidates or to restrict the search. We prove consistency with the observed examples, identification within bounded hypothesis classes with a bound on the number of queries, and a PAC guarantee for the final hypothesis. In experiments with a simulated teacher on 36 targets, including targets in four standard environments, the learner identifies every target when it compares all rivals, with fewer than seven queries beyond an initial sample of ten traces on average. Correct hints reduce the number of queries by up to 16% and the number of generated candidates by a fifth, and hints used for ranking never prevent identification, even when incorrect. Our enumeration-based version space is faster than the SAT-based approach of previous learners at every target size.**

I. Introduction

Temporal logics are ubiquitous in formal methods, providing a compact and interpretable language for expressing system requirements in domains including verification [27], [37], runtime monitoring [5], and planning [3]. Recently, they have been employed in the verification of machine learning systems [32], [17], agentic frameworks [38], [18], and robotic systems [19].

However, formalising the intended behaviour in a temporal logic can be a complex and error-prone process, in particular for non-experts [16]. Active learning provides a framework for identifying correct specifications through interaction with a teacher who is assumed to know the target specification. In a typical exact-learning loop, the learner poses membership queries, asking whether a specific execution satisfies the target specification, and equivalence queries, asking whether the current hypothesis is equivalent to the target. For a failed equivalence query, the teacher provides a counterexample on which the hypothesis and target disagree. The learner uses this feedback to refine its hypothesis [1], [2], [35].

This interaction places substantial demands on the teacher, particularly when answering equivalence queries and constructing counterexamples, and prevents non-expert users from serving as teachers. Users may understand the intended behaviour well enough to judge whether an execution is acceptable or to explain their concerns in natural language, yet lack the formal expertise needed to assess an entire specification. This gap limits the applicability of equivalence-query-based approaches to user-facing specification acquisition and motivates learning methods that rely on execution labels and informal guidance.

Existing active learners for temporal logics share a second limitation. They encode the search for a consistent formula as a SAT or optimisation problem [7], [16], following SAT-based passive learning [26]. Such a learner either solves the problem anew after each example [7], or computes a small set of candidates once and may lose the target [16]. Since a formula has many encodings, every solution must be blocked encoding by encoding, and the search becomes slow as the class of formulas grows.

We present a modular active-learning framework for LTLf in which the teacher only labels traces and, optionally, answers natural-language questions. The framework never asks the teacher to judge a formula or to construct a counterexample. The learner keeps a *version space* of the candidate formulas consistent with the labels, compares the current hypothesis with rivals from this version space, and asks the teacher to label a trace on which the two formulas disagree. Each of these steps is a separate procedure that can be replaced independently. Our learner enumerates the formulas of a bounded class once, with the simplifications of grammar-based enumeration [31], and finds distinguishing traces with a sound and complete procedure, either an automaton construction [11] or formula progression [3]; the SAT-based version space of previous learners fits the same loop. Each label removes every inconsistent candidate at once, so a few queries suffice even when the class contains millions of formulas. A rival for which no such trace exists is equivalent to the hypothesis and is removed without a query. In an environment, the search is restricted to feasible traces.

Membership labels state whether an execution is acceptable, but not why. Our framework therefore also asks *natural-language queries*. At Level 1, the user selects the kind of problem, such as a missing event or a wrong ordering; at Level 2, the user states a requirement of the intended behaviour, which is translated into a formula, for instance by a large language model. The learner uses these hints to focus the search on compatible formulas.

Once a hypothesis is selected, fresh, independently sampled conformance tests assess its agreement with the teacher under a specified distribution. This gives a statistical guarantee on unseen behaviour without requiring the teacher to establish

logical equivalence.

Our contributions are:

- an active-learning framework for LTLf that uses membership queries and natural-language hints instead of equivalence queries, and in which the version space, the order of rivals and the distinguishing procedure can each be replaced (Section IV);
- two levels of natural-language hints that focus the search on compatible formulas (Section IV-D);
- the generation of distinguishing queries by automata or by LTLf progression, also restricted to the feasible traces of an environment (Section IV-C);
- guarantees of sample consistency, identification within bounded hypothesis classes with a bound on the number of queries, and a PAC guarantee for the final hypothesis (Section V);
- an evaluation with $5{,}540$ runs on 20 targets over free traces and 16 targets in four standard environments, including a comparison with automata-based distinguishing and with the SAT-based version space of previous learners (Section VII).

In our experiments with a simulated teacher, the learner identifies every target in every run when it compares all rivals, with on average 6.2 queries beyond an initial sample of 10 traces over free traces and 2.5 in environments; from size-2 to size-7 targets, the number of queries grows from 11 to 21, while the number of generated formulas grows from 200 to 3.9 million. Correct hints reduce the number of queries by up to 16% and the number of generated candidates by up to a fifth. With a fixed shortlist of three rivals, they raise the share of environment runs that identify the target from 50% to 80%. Incorrect hints can mislead the learner (Section VII-D). Compared with our reimplementation of the SAT-based version space of previous learners, used in the same loop, our version space is faster at every target size (8.8 s against 22.4 s for size-7 targets), and it completes searches that the SAT-based version space does not finish within ten minutes.

A. Motivating Example

Consider a mobile service robot that picks up objects and delivers them. The user knows the intended behaviour but cannot write it in temporal logic. Over $\mathsf{AP} = \{pickup, deliver\}$, the target is $\varphi_T = \mathbf{G}\big(pickup \rightarrow \mathbf{X}(\neg pickup\, \mathbf{U}\, deliver)\big)$: every pickup is followed later by a delivery, with no other pickup in between. We follow a run from our experiments with a simulated teacher.

First, the user labels two executions: $\{pickup\};\{deliver\}$ is accepted and $\{pickup, deliver\}$ is rejected. Asked whether something required is missing, the user answers that the object should be delivered after a pickup; this hint becomes the requirement $h = \mathbf{G}(pickup \rightarrow \mathbf{F}\, deliver)$, and the learner first searches the formulas consistent with it. Then, the learner repeatedly compares its hypothesis with other consistent formulas and asks the user about an execution on which they disagree. For instance, the user rejects $\{pickup\};\{pickup\};\{deliver\}$, which refutes the hypothesis $\mathbf{G}(pickup \rightarrow \mathbf{FX}\, deliver)$ and makes φ_T the new hypothesis. Finally, when no remaining formula can be told apart from the hypothesis, the learner returns φ_T, after 16 membership queries including the two initial ones; without the hint, it needs 20.

a) Related work.: Active learning originates in automata learning, with Angluin's L^* algorithm [1] and its use for behavioural models of software and hardware systems [35]. Three works learn temporal formulas actively and are closest to ours. Camacho and McIlraith [7] learn a minimal LTLf formula with equivalence queries: a SAT-based passive learner computes a smallest formula consistent with the examples, and each counterexample returned by the oracle is added to the examples before the passive learner runs again. Linard and Tůmová [22] learn STL specifications with membership and equivalence queries. Their learner keeps an online decision tree as its hypothesis, asks for labels of signals generated from the hypothesis, and tests equivalence by a distance to the target; a failed equivalence query returns a labelled counterexample. Gavran et al. [16] synthesise LTL specifications of robot tasks from one example trace and a natural-language description. An optimisation-modulo-theories encoding generates a small set of candidate specifications, and the user states whether behaviours that distinguish them match the intended task.

Our framework differs from these works in three respects. First, the teacher only labels traces and answers questions. Unlike in [7], [22], the teacher never judges a hypothesis or constructs a counterexample. Second, the learner keeps every consistent formula of the bounded class, so it identifies the target with a bounded number of queries (Section V). A small set of candidates [16] may not contain the target, and for a decision-tree hypothesis [22], convergence is observed experimentally but not proved. Third, our version space is computed once and only filtered as labels arrive, whereas the learner of [7] solves a SAT problem anew after each counterexample. In our experiments, our version space is faster than a SAT-based one at every target size. Natural language also plays a different role. In [16], it describes the task at the start; our natural-language queries elicit partial requirements during learning, alongside membership labels. Finally, our procedures can each be replaced; for example, a SAT-based version space needs less memory for larger classes (Section VII-C).

Passive learning infers a formula from a fixed set of labelled examples, without querying a teacher. For LTL and LTLf, existing approaches include SAT-based inference [26], scalable anytime learning [29], graph-neural-network-based inference [24], learning from positive examples [30], statistical inference from event logs [9], and grammar-based enumeration with search heuristics [31], which our version space adopts. For STL, researchers have developed decision-tree methods [6], neural-symbolic approaches [20], and optimisation-based decision-tree learning [21]; Baharisangari et al. [4] use STL inference for causal discovery from time-series data. Other work translates natural-language commands into temporal specifications with large language models [23]; it starts from a complete description of the task, whereas our learner combines labels with partial requirements.

II. Preliminaries

A. Linear Temporal Logic on Finite Traces

To illustrate the framework, we use linear temporal logic on finite traces (LTLf) and introduce the necessary definitions [11]. We represent relevant system properties by a finite set of atomic propositions AP. The alphabet $\Sigma = 2^{\mathsf{AP}}$ consists of all subsets of AP; each letter specifies which propositions hold at a given position. We use the trace domain $\mathcal{W} = \Sigma^*$, which includes the empty trace ε.

The syntax of LTLf formulas is defined by the grammar

$$\varphi ::= p \in \mathsf{AP} \mid \neg\varphi \mid \varphi \wedge \varphi \mid \mathbf{X}\varphi \mid \varphi\,\mathbf{U}\,\varphi.$$

We also use the standard derived Boolean operators $\vee$ and $\rightarrow$, constants false and true, and the temporal operators $\mathbf{F}$ (eventually) and $\mathbf{G}$ (always).

For a finite trace w and an index $0 \le i \le |w|$, satisfaction is defined as follows:

$$\begin{aligned}
w, i &\models p && \text{iff } i < |w| \text{ and } p \in w_i,\\
w, i &\models \neg\varphi && \text{iff } w, i \not\models \varphi,\\
w, i &\models \varphi \wedge \psi && \text{iff } w, i \models \varphi \text{ and } w, i \models \psi,\\
w, i &\models \mathbf{X}\varphi && \text{iff } i+1 < |w| \text{ and}\\
& && \quad w, i+1 \models \varphi,\\
w, i &\models \varphi\,\mathbf{U}\,\psi && \text{iff there is } j \ge i \text{ with } j < |w|,\ w, j \models \psi,\\
& && \quad \text{and } w, k \models \varphi \text{ for every } i \le k < j.
\end{aligned}$$

We write $w \models \varphi$ for $w, 0 \models \varphi$ and $L(\varphi) = \{w \in \mathcal{W} \mid w \models \varphi\}$. We define equivalence and implication through the corresponding languages: $\varphi \equiv \psi$ means $L(\varphi) = L(\psi)$, and $\varphi \models \psi$ means $L(\varphi) \subseteq L(\psi)$.

Following [31], we measure formula complexity by size and temporal depth. The size $|\varphi|$ is the number of syntax-tree nodes, counting each literal (including negated propositions) as a single leaf. The temporal depth $\mathrm{td}(\varphi)$ is the maximum number of temporal operators along any root-to-leaf path. For example, $\mathbf{G}(p \rightarrow \mathbf{F}(\neg q))$ has size 5 and temporal depth 2.

B. Background on Active Learning

Active learning [1] involves two entities: a learner and a teacher. The teacher represents a non-expert user who knows the intended target specification φ_T and responds to the learner's queries. The learner interacts with the teacher through two types of queries, formalised below: membership queries, which request the teacher to classify traces according to the target specification, and natural-language queries, which elicit hints to guide the refinement of the learned specification. Note that, in contrast to traditional active learning, our framework avoids equivalence queries, as non-expert users cannot be expected to assess whether a learned specification precisely captures their intentions.

Definition 1 (Membership query)**.** *Given a target φ_T and a trace $w \in \mathcal{W}$, a membership query $\mathsf{MQ}(w)$ returns 1 if $w \models \varphi_T$ and 0 otherwise.*

Definition 2 (Natural-language query)**.** *Given a target φ_T, a natural-language query returns a hint h describing a property of φ_T.*

The traces labelled so far and their labels form a labelled *sample* $S \subseteq \mathcal{W} \times \{0, 1\}$. A formula φ is *consistent* with S if, for every $(w, y) \in S$, $w \models \varphi$ if and only if $y = 1$.

III. Learning Problem

Fix a finite set of propositions AP and the trace domain $\mathcal{W} \subseteq (2^{\mathsf{AP}})^*$. Let $\mathcal{L}_{n,d}$ denote the finite class of LTLf formulas over AP with size at most n and temporal depth at most d (Section IV-A). The teacher T answers membership queries according to an unknown target $\varphi_T \in \mathcal{L}_{N,D}$, where N, D are unknown to the learner L.

The objective is to identify a hypothesis φ_H using as few membership queries as possible, optionally guided by natural-language hints. Every hypothesis must be consistent with the labelled sample $S = \{(w, \mathsf{MQ}(w)) \mid w \text{ was queried}\}$. We distinguish two objectives: (i) *exact identification*, $\varphi_H \equiv \varphi_T$, when the searched class contains φ_T and refinement is exhaustive; and (ii) *approximate identification* under a distribution $\mathcal{D}$ over $\mathcal{W}$, measured by

$$\mathrm{err}_{\mathcal{D}}(\varphi_H, \varphi_T) = \Pr_{w \sim \mathcal{D}}[\mathbf{1}[w \models \varphi_H] \neq \mathbf{1}[w \models \varphi_T]].$$

For $\epsilon, \delta \in (0, 1)$, certification must satisfy

$$\Pr[\text{accept } \varphi_H \ \wedge\ \mathrm{err}_{\mathcal{D}}(\varphi_H, \varphi_T) > \epsilon] \le \delta.$$

This guarantee does not imply exact equivalence.

IV. Active Learning Framework

Before giving the details, we outline the structure of the framework. The learner starts from an initial hypothesis and refines it iteratively, using several interchangeable procedures, which we describe briefly below.

Initialize(L) labels L traces (line 1): random traces whose length is drawn uniformly from a fixed range and whose letters are drawn uniformly from Σ, feasible rollouts in an environment, or a given sample. The learner then searches a chain of finite classes $\mathcal{L}_1 \subseteq \cdots \subseteq \mathcal{L}_M$, by default $\mathcal{L}_m = \mathcal{L}_{m,d}$ for a fixed depth bound d, so that smaller formulas come first. For each class, VersionSpace adds the new formulas consistent with S to the version space V (line 4; Section IV-A), and the first hypothesis is a smallest formula of V (line 6). AskHint may ask the teacher once for hints, which reorder the rivals or replace the class schedule (line 7; Section IV-D). In each refinement round (lines 8–21), NextRival orders the other formulas of V, called *rivals* (Section IV-B), and the learner compares φ_H with the first K of them. If Distinguish (Section IV-C) finds no trace, the rival is equivalent to φ_H and is removed without a query (line 13). Otherwise, the teacher labels the returned trace w (line 15). If the label refutes φ_H, the rival, which agrees with the label, becomes the hypothesis (line 16), and every formula inconsistent with the label leaves V (line 17). A round without a query ends refinement on the current class. Finally, Certify tests the hypothesis on P fresh

Algorithm 1 Active learning by distinguishing queries

Require: Class schedule $\mathcal{L}_1 \subseteq \cdots \subseteq \mathcal{L}_M$, shortlist size $K \in \mathbb{N} \cup \{\infty\}$, budgets L, P
Ensure: Hypothesis φ_H, labelled sample S, certification result $(b, \widehat{e})$

1: $S \leftarrow$ INITIALIZE(L)
2: $\mathcal{H}_{\text{hint}} \leftarrow \emptyset$; $\varphi_H \leftarrow$ false; $V \leftarrow \emptyset$; $\mathcal{L}_0 \leftarrow \emptyset$
3: **for** $m = 1, \ldots, M$ **do**
4: $\quad V \leftarrow V \cup$ VERSIONSPACE$(\mathcal{L}_m \setminus \mathcal{L}_{m-1}, S)$
5: $\quad$ **if** $V = \emptyset$ **then continue**
6: $\quad$ **if** $\varphi_H =$ false **then** $\varphi_H \leftarrow$ a smallest formula in V
7: $\quad \mathcal{H}_{\text{hint}} \leftarrow \mathcal{H}_{\text{hint}} \cup$ ASKHINT(φ_H) ▷ Optional, at most once
8: $\quad$ **repeat**
9: $\quad\quad R \leftarrow$ the first K formulas of NEXTRIVAL$(V, \varphi_H, \mathcal{H}_{\text{hint}})$
10: $\quad\quad$ **for all** $\psi \in R$ **do**
11: $\quad\quad\quad w \leftarrow$ DISTINGUISH(φ_H, ψ)
12: $\quad\quad\quad$ **if** $w =$ false **then**
13: $\quad\quad\quad\quad V \leftarrow V \setminus \{\psi\}$ ▷ $\psi \equiv \varphi_H$
14: $\quad\quad\quad$ **else**
15: $\quad\quad\quad\quad S \leftarrow S \cup \{(w, \mathsf{MQ}(w))\}$
16: $\quad\quad\quad\quad$ **if** φ_H is inconsistent with S **then** $\varphi_H \leftarrow \psi$
17: $\quad\quad\quad\quad V \leftarrow \{\varphi \in V \mid \varphi$ is consistent with $S\}$
18: $\quad\quad\quad\quad$ **break** ▷ Start the next round
19: $\quad\quad\quad$ **end if**
20: $\quad\quad$ **end for**
21: $\quad$ **until** no query was asked in this round
22: **end for**
23: **if** $\varphi_H =$ false **then return** (FAILURE, S)
24: $(b, \widehat{e}) \leftarrow$ CERTIFY(φ_H, P)
25: **return** $(\varphi_H, S, b, \widehat{e})$

traces (line 24; Section IV-E). The implementation also bounds the number of comparisons (Section VI).

a) Correctness and cost.: Equivalence is over $\mathcal{W}$, or relative to an environment. A query separates φ_H from the compared rival, so exactly one of them agrees with $\mathsf{MQ}(w)$; the other one, and every formula equivalent to it, leaves V, while the class of φ_T is never removed. With $K = \infty$ (the *exhaustive rule*), a complete DISTINGUISH and $\varphi_T \in \mathcal{L}_M$, refinement on $\mathcal{L}_M$ ends only when every remaining formula is equivalent to φ_T (Section V). With a finite K (the *top-K rule*), refinement can end while a rival that is not equivalent to φ_H remains outside the shortlist. The version space and the order of rivals affect the number of queries and the running time, but not the correctness of the result.

A. Version space

For a class $\mathcal{L}$ and a sample S, the version space is

$$V_{\mathcal{L}}(S) = \{\psi \in \mathcal{L} \mid w \models \psi \text{ iff } y = 1 \text{ for all } (w, y) \in S\}.$$

VERSIONSPACE$(\mathcal{L}, S)$ returns $V_{\mathcal{L}}(S)$. Algorithm 1 calls it only for the formulas that are new in the schedule (line 4), and it keeps the version space up to date after each label (line 17). We describe two implementations.

a) Enumeration.: Following the grammar-based enumeration of Roy et al. [31], in the first approach, we build the version space bottom-up by dynamic programming. Starting from the atomic propositions and their negations, the formulas of each size and nesting depth are built from smaller ones with the operators $\mathbf{X}$, $\mathbf{F}$, $\mathbf{G}$, $\mathbf{U}$, $\wedge$, $\vee$ and $\rightarrow$. A new formula enters the version space only if it is consistent with the current S.

As in [31], two heuristics discard redundant formulas. *Temporal simplification* discards the forms $\mathbf{FF}\varphi$, $\mathbf{GG}\varphi$, $\mathbf{X}\varphi\,\mathbf{U}\,\mathbf{X}\psi$, $\mathbf{G}\varphi \wedge \mathbf{G}\psi$, $\mathbf{X}\varphi \wedge \mathbf{X}\psi$, $\mathbf{F}\varphi \vee \mathbf{F}\psi$ and $\mathbf{X}\varphi \vee \mathbf{X}\psi$, each equivalent over finite traces to a smaller formula. *Boolean simplification* discards binary formulas whose operands are syntactically equal or complementary literals. Since larger formulas are built from smaller ones, all formulas of size below n are stored, consistent or not, so time and memory grow with $|\mathcal{L}_{n,d}|$.

b) SAT-based construction.: Our second approach follows SAT-based passive learning [26], [7] and represents the version space implicitly by a propositional formula $\Phi^S_{m,d} = \phi_{\text{syn}} \wedge \phi_{\text{sem}}$, whose satisfying assignments encode the formulas of size m and temporal depth at most d that are consistent with S: ϕ_{syn} encodes their syntax and ϕ_{sem} their semantics on the sample traces.

For ϕ_{syn}, a formula of size m is represented by its syntax tree with nodes $1, \ldots, m$: the root is node m, children have smaller indices than their parents, leaves are propositions or their negations, and inner nodes are operators. The variable $x_{i,\lambda}$ indicates that node i is labelled by the literal or operator λ, and $l_{i,j}$ and $r_{i,j}$ indicate that node j is the left or right child of node i. The constraints assign exactly one label to each node and give each operator as many children as its arity. Unlike the syntax DAGs of [26], every non-root node has exactly one parent, so the encoding represents syntax trees and matches our size measure. In addition, the variable $e_{i,k}$ indicates that the subformula rooted at node i has temporal depth at most k, and the constraints propagate these values from the leaves to the root, so that the root has temporal depth at most d.

For ϕ_{sem}, each trace w of S is evaluated position by position. The variable $z^w_{i,k}$ indicates that the subformula rooted at node i holds at position k of w, for $0 \le k \le |w|$, where $k = |w|$ is the end position. The constraints relate these variables to the labels and children of the nodes, following the finite-trace semantics of the preliminaries, and require $z^w_{m,0} = y$ for each $(w, y) \in S$. Thus $\Phi^S_{m,d}$ is satisfiable if and only if some formula of size m and temporal depth at most d is consistent with S, and such a formula can be decoded from any satisfying assignment.

In this approach, the version space need not be listed explicitly. NEXTRIVAL asks an incremental SAT solver for models of $\Phi^S_{m,d}$, in increasing order of m. If DISTINGUISH proves the decoded rival ψ equivalent to φ_H, a blocking clause, as in [26], [16], excludes the current assignment to the structural variables $x_{i,\lambda}$, $l_{i,j}$ and $r_{i,j}$.

B. Next rival

NEXTRIVAL$(V, \varphi_H, \mathcal{H}_{\text{hint}})$ lists the formulas of $V \setminus \{\varphi_H\}$ in some order, and Algorithm 1 compares φ_H with the first K of them. The order decides which formulas are compared, and hence which traces are queried. We consider three orders.

a) Enumeration.: The rivals come in the order in which the version space produces them: smallest formulas first, with ties broken by a fixed textual order. With the SAT-based construction, this is the order in which the solver returns models, again smallest size first. The order needs no extra computation.

b) Semantic.: The rivals closest to φ_H in behaviour come first. For a probe set $U = \{u_1, \ldots, u_r\}$, the semantic distance

$$d_{\text{sem}}(\varphi, \psi; U) = \frac{1}{r} \sum_{i=1}^{r} \mathbf{1}[\mathbf{1}[u_i \models \varphi] \neq \mathbf{1}[u_i \models \psi]]$$

is the fraction of probe traces on which the two formulas disagree. The probe set is drawn once per run, independently of the sample, from the distribution of the initial traces, or as feasible rollouts in an environment, so d_{sem} estimates the probability that the two formulas disagree on such a trace. We also evaluate the reverse order, *semantic-far*, which puts the farthest rivals first.

c) Split.: The rivals whose witness divides the version space most evenly come first. Let w_ψ be the first trace in the pool T_ℓ of short traces (Section IV-C) on which ψ and φ_H disagree. Rivals are sorted by $\big|2\,|\{\varphi \in V \mid w_\psi \models \varphi\}| - |V|\big|$, so the rivals whose witness comes closest to halving the version space are compared first; whatever its label, such a witness removes many formulas. Rivals with no witness in T_ℓ follow in enumeration order.

C. Distinguish

DISTINGUISH(φ, ψ) returns a trace of $\mathcal{W}$ on which φ and ψ disagree, or false. For our framework, one witness per comparison suffices: whatever its label, one of the two formulas becomes inconsistent with the sample. We give two sound and complete procedures, which we describe below.

1) Distinguishing with automata.: The first procedure translates each LTLf formula φ into a deterministic finite automaton (DFA) $\mathcal{A}_\varphi$ with the same language [11]. For candidates φ and ψ, the product construction [28], in which a state is accepting exactly when one of its components is, yields a DFA $\mathcal{A}_\Delta$ with $L(\mathcal{A}_\Delta) = L(\mathcal{A}_\varphi) \triangle L(\mathcal{A}_\psi)$. Every trace of $L(\mathcal{A}_\Delta)$ is accepted by exactly one candidate, so the procedure returns such a trace if $\mathcal{A}_\Delta$ is nonempty, and false otherwise.

2) Distinguishing by formula progression.: The automata-based procedure builds an automaton for every compared formula. Our implementation instead explores the symmetric-difference automaton on the fly, using formula progression [3]. For a letter a, the progression $\text{prog}(\varphi, a)$ is the formula that the rest of the trace must satisfy once a has been read; it extends to traces letter by letter. The *end value* $\text{fin}(\varphi) \in \{0, 1\}$ states whether φ holds on the empty trace. Both are computed syntactically, and $uw \models \varphi$ if and only if $w \models \text{prog}(\varphi, u)$. Appendix B gives the definitions.

Example 1. *Let* $\mathsf{AP} = \{a, b\}$ *and* $\varphi = a\,\mathbf{U}\,b$*. Then* $\text{prog}(\varphi, \{a\}) = \varphi$*,* $\text{prog}(\varphi, \{b\}) = \text{true}$ *and* $\text{prog}(\varphi, \emptyset) = \text{false}$*, while* $\text{fin}(\varphi) = 0$ *and* $\text{fin}(\text{true}) = 1$*. Hence* $\{a\}\{b\} \models \varphi$*, and* $\{a\} \not\models \varphi$*. To compare* φ *with* $\psi = \mathbf{F}b$*, for which* $\text{prog}(\psi, \emptyset) = \psi$ *and* $\text{prog}(\psi, \{b\}) = \text{true}$*, the trace* $\emptyset\{b\}$ *progresses the pair* (φ, ψ) *to* $(\text{false}, \text{true})$*, whose end values differ, while no shorter trace distinguishes* φ *from* ψ*.*

The procedure is a breadth-first search over pairs of progressed formulas. It starts from (φ, ψ) with the empty trace. From a pair reached by a trace u, each letter a leads to the pair progressed by a, reached by ua. The search returns u as soon as the two end values differ. It visits each pair once and does not expand a pair with equal components, since no extension of u distinguishes the formulas there. We keep progressed formulas in a *normal form*, as sets of clauses over the subformulas of the input, so only finitely many pairs arise and the search terminates. In the worst case it visits doubly exponentially many pairs in $|\varphi| + |\psi|$; in practice it builds only the pairs it reaches and stops at the first witness. The implementation bounds the number of pairs (Section VI); a search that exceeds the bound reports a failure instead of an answer.

Lemma 1 (Correctness of DISTINGUISH)**.** *Let* φ *and* ψ *be LTLf formulas, and let* DISTINGUISH *be the automata-based or the progression-based procedure, without a budget. Then* DISTINGUISH(φ, ψ) *terminates, and it returns a trace if and only if some* $w \in \Sigma^*$ *satisfies* $(w \models \varphi) \oplus (w \models \psi)$*; every returned trace satisfies this condition. Hence it returns* false *if and only if* φ *and* ψ *are equivalent over* Σ^**. The progression-based procedure returns a shortest such trace.*

The proof is given in Appendix C.

a) Environments.: For an environment given as a finite labelled transition system, the progression-based search runs over triples of an environment state and two progressed formulas, and follows the transitions of the environment instead of all letters. There are finitely many triples, so the search returns a nonempty feasible distinguishing trace if and only if one exists. During learning, the implementation also bounds the number of actions (Section VI); a search that reaches this bound returns no answer, which preserves soundness but not completeness.

D. Natural-language hints

A hint gives the learner a *partial requirement*: a property that every acceptable execution has, but that need not describe the whole target. Hints never change a label. They act on Algorithm 1 in two ways only: by reordering the rivals in NEXTRIVAL, or by restricting the class schedule of VERSION-SPACE. An execution shown to the teacher may prompt a hint, but the learner uses only the answer. The learner asks at two levels (Table I): at Level 1 the teacher selects a question type c, and at Level 2 they state a requirement, which is translated into an LTLf formula h.

TABLE I

QUESTIONS AT BOTH LEVELS AND THE FORMULA FAMILIES $\mathcal{H}_c$ OF THE QUESTION TYPES. LEVEL 1 ANSWERS ARE YES, NO, OR UNSURE; LEVEL 2 IS ASKED AFTER A YES ANSWER. B AND B' STAND FOR BOOLEAN FORMULAS. IN OUR EXPERIMENTS, A SIMULATED TEACHER ANSWERS.

Type	Level 1 question	Level 2 question	Patterns in $\mathcal{H}_c$
Undesired behaviour	Does anything happen that should not happen?	Which event should not happen, and when?	$\mathbf{G}B$, $\mathbf{G}\neg B$, $\mathbf{G}\neg(B \wedge B')$
Missing behaviour	Is something required missing?	Which event is missing, and when is it required?	$\mathbf{F}B$, $\mathbf{G}(B \rightarrow \mathbf{F}B')$, $\mathbf{G}(B \rightarrow \mathbf{XF}B')$
Ordering	Do any events happen in the wrong order?	Which events, and in what order?	$B\,\mathbf{U}\,B'$, $\mathbf{F}(B \wedge \mathbf{F}(B' \wedge \cdots))$, responses, $\mathbf{G}(B \rightarrow \mathbf{X}B')$
Completion	Is any task left unfinished?	Which task, and what would complete it?	$\mathbf{F}B$, responses, $\mathbf{FG}B$
Repetition	Does anything happen too often or not often enough?	Which event, and how should its repetition change?	$\mathbf{GF}B$

a) Level 1: question types.: Each question type c stands for a set of specification patterns [13], and $\mathcal{H}_c$ is the family of formulas that are instances of these patterns (Table I). For example, missing behaviour stands for eventuality and response, with instances such as $\mathbf{F}a$ and $\mathbf{G}(b \rightarrow \mathbf{F}a)$. The families overlap, since one requirement can be described by different question types. The learner uses only the question types answered with Yes.

b) Level 2: a containing language.: The Level-2 answer names the events and conditions of a requirement. It is translated into a formula h that every acceptable execution should satisfy, for example by a large language model; we assume the translation is given and do not evaluate it. A correct hint satisfies $\varphi_T \models h$, that is, $L(\varphi_T) \subseteq L(h)$. For example, "every pickup must be followed by a delivery" gives $h = \mathbf{G}(\mathit{pickup} \rightarrow \mathbf{F}\,\mathit{deliver})$. This hint is correct for the target of the motivating example (Section I-A), but it does not determine that target.

c) Using hints.: ASKHINT (line 7 of Algorithm 1) asks the Level-1 question at most once in a run. The families $\mathcal{H}_c$ of the question types answered with Yes are the *hinted families*. The learner uses them in one of two ways.

Ranking. NEXTRIVAL sorts the rivals stably by their distance to the hinted families: the smallest number of syntactic changes that turn the rival into an instance of a hinted family. Ranking changes only the order in which rivals are compared.

Pruning. The class schedule is replaced by levels: first the formulas of $\mathcal{L}_{n,d}$ in the hinted families, then those whose root operator is the root operator of some formula in a hinted family, and finally $\mathcal{L}_{n,d}$. Each level is searched by size, as in the default schedule. A Level-2 hint h adds a first level that contains the formulas ψ rejecting every trace of T_ℓ that h rejects, which is the check $\psi \models h$ restricted to T_ℓ. The learner moves to the next level only when the current level leaves the version space empty. Since pruning changes the class schedule, its hints are asked before the first call of VERSIONSPACE (line 4) rather than at line 7.

Ranking excludes no candidate, so an incorrect ranking hint can only lead to less useful queries (Lemma 2). Pruning excludes candidates: if an incorrect hint leaves a consistent formula in a level without φ_T, the learner ends in that level.

Lemma 2 (Robustness to inaccurate hints)**.** *Suppose every membership-query answer is labelled according to φ_T, and hints affect only candidate ranking and query selection. Then, regardless of the accuracy of the question types c and their interpreted requirements h, every hypothesis selected from a nonempty version space is consistent with the accumulated sample S.*

The proof is given in Appendix C.

E. Certify

We assess distributional error in the probably approximately correct learning framework [36], using a binomial test of a fixed hypothesis [10]. The final hypothesis is frozen. CERTIFY(φ_H, P) draws up to P fresh traces i.i.d. from a declared distribution $\mathcal{D}$ and asks their labels by membership queries. It fails as soon as a test disagrees with φ_H, and returns that trace as a counterexample. If all P tests agree, it succeeds, and the guarantee of Theorem 3 applies.

V. FORMAL PROPERTIES

The identification results concern Algorithm 1 with the exhaustive rule ($K = \infty$), the class schedule by size, no hint pruning, and a comparison budget that is not exhausted; $\mathcal{H}$ denotes the largest searched class $\mathcal{L}_M$, and $\mathcal{H}/\equiv$ its set of equivalence classes. Proofs are in Appendix C.

Proposition 1 (Sample consistency)**.** *Any hypothesis returned from a nonempty version space agrees with all labels in S.*

Theorem 1 (Conditional bounded identification)**.** *Let $\varphi_T \in \mathcal{H}$, and suppose* DISTINGUISH *returns a witness for every pair of non-equivalent formulas of $\mathcal{H}$. Then Algorithm 1 asks at most $|\mathcal{H}/\equiv| - 1$ refinement queries (line 15) and ends with $\varphi_H \equiv \varphi_T$.*

Under either stopping rule, every query removes at least one equivalence class (Section IV), so a run asks at most $L + |\mathcal{H}/\equiv| - 1 + P$ membership queries, including the initial sample and certification; the comparison budget bounds comparisons, not queries. The implementation bounds DISTINGUISH by a number of product states and, in environments, by a horizon (Section VI); in our experiments, these bounds did not affect any final hypothesis (Section VII-B).

a) Identification relative to an environment.: In an environment $\mathcal{E}$, queries are restricted to its feasible traces $L(\mathcal{E}) \subseteq \Sigma^+$, so formulas that differ only on infeasible traces cannot be told apart.

Definition 3 (Environment-relative equivalence)**.** *Two formulas φ and ψ are equivalent relative to an environment $\mathcal{E}$, written $\varphi \equiv_{\mathcal{E}} \psi$, if $L(\mathcal{E}) \cap \big(L(\varphi) \triangle L(\psi)\big) = \emptyset$.*

Equivalence implies $\equiv_{\mathcal{E}}$, but not conversely, so $N_{\mathcal{E}} = |\mathcal{H}/\equiv_{\mathcal{E}}| \leq |\mathcal{H}/\equiv|$. DISTINGUISH then searches for a witness in $L(\mathcal{E}) \cap (L(\varphi) \triangle L(\psi))$.

Theorem 2 (Conditional environment-relative identification)**.** *Let $\varphi_T \in \mathcal{H}$, and suppose* DISTINGUISH *returns a feasible witness for every pair of formulas of $\mathcal{H}$ that are not equivalent relative to $\mathcal{E}$. Then Algorithm 1 asks at most $N_{\mathcal{E}} - 1$ refinement queries and ends with $\varphi_H \equiv_{\mathcal{E}} \varphi_T$.*

The environment can thus reduce the number of queries. The guarantee needs a complete search over feasible traces: if feasible traces are only sampled, failing to find a witness does not establish $\equiv_{\mathcal{E}}$.

b) Certification.: The following bound inverts the binomial probability of observing no errors, as in exact binomial confidence limits [10].

Theorem 3 (PAC guarantee)**.** *Let φ_H be fixed before P tests are drawn i.i.d. from $\mathcal{D}$. With probability at least $1-\delta$ over the tests, some test disagrees with φ_H, or $\mathrm{err}_{\mathcal{D}}(\varphi_H, \varphi_T) \leq \ln(1/\delta)/P$.*

The same holds for a distribution $\mathcal{D}_{\mathcal{E}}$ supported on $L(\mathcal{E})$, with $\mathrm{err}_{\mathcal{D}_{\mathcal{E}}}(\varphi_H, \varphi_T) = \Pr_{w \sim \mathcal{D}_{\mathcal{E}}}[(w \models \varphi_H) \neq (w \models \varphi_T)]$. Unlike Theorem 2, this bound needs no exhaustive exploration of the environment, but it bounds the disagreement under $\mathcal{D}_{\mathcal{E}}$, not $\equiv_{\mathcal{E}}$.

VI. IMPLEMENTATION

We implemented Algorithm 1 for LTLf in Python 3.14. Each procedure of Section IV is a separate module, so the enumeration-based and the SAT-based learner share every procedure except VERSIONSPACE. The code and all experiments are available at https://anonymous.4open.science/r/ltl-active-learning-with-hints-585D/. DISTINGUISH first evaluates both formulas on the pool T_ℓ of at most 5,000 short traces and returns the first trace on which they disagree; this saves time and gives the same trace as the complete search (Section IV-C). Otherwise, the progression-based search runs, bounded by 10^5 product states and, in environments, by 30 actions; a comparison that reaches this horizon is undecided, and the rival is skipped. For the automata-based search, we build the automata with LTLf2DFA 1.0.2 [15] and MONA 1.4 [14].

All experiments ran on an Apple M4 Pro with 24 GB of memory. The solver is the incremental SAT engine of Z3 5.3.3 [25], used through its Python API; a run is halted after $2 \cdot 10^5$ solver calls or 600 s. The environments use Gymnasium 1.4.0 [34] and MiniGrid 3.2.0 [8].

VII. EXPERIMENTAL EVALUATION

Our evaluation studies the practical performance of the learner: whether it identifies the target, what identification costs in queries and time, and how natural-language hints change both. All experiments concern LTLf over propositional traces.

TABLE II
BENCHMARK TARGETS. $|S|$: NUMBER OF REACHABLE ENVIRONMENT STATES. SIZES COUNT SYNTAX-TREE NODES, WITH $\neg p$ COUNTED AS ONE NODE.

Free traces, $\mathsf{AP} = \{a, b, c\}$	
Size 2	$\mathbf{F}a$, $\mathbf{G}a$
Size 3	$\mathbf{FG}a$, $\mathbf{GF}a$, $a\,\mathbf{U}\,b$
Size 5	$\mathbf{F}a \to \mathbf{F}b$, $\mathbf{G}(a \to \mathbf{F}b)$, $\mathbf{G}(a \to \mathbf{X}b)$, $\mathbf{F}(a \wedge \mathbf{F}b)$, $a\,\mathbf{U}\,(b\,\mathbf{U}\,c)$, $\mathbf{F}a \wedge \mathbf{G}\neg b$, $\mathbf{F}(a \wedge \mathbf{X}b)$
Size 6	$\mathbf{G}(a \to b\,\mathbf{U}\,c)$, $a \to \mathbf{X}(b\,\mathbf{U}\,c)$, $\neg a\,\mathbf{U}\,(b \wedge \mathbf{F}c)$, $\mathbf{G}(\neg a \vee \mathbf{XG}\neg b)$
Size 7	$\mathbf{G}((a \wedge b) \to \mathbf{X}c)$, $\mathbf{GF}a \wedge \mathbf{GF}b$, $\mathbf{G}(a \to \mathbf{X}(a\,\mathbf{U}\,b))$, $\mathbf{F}(a \wedge \mathbf{F}(b \wedge c))$
Environments	
Office, $\lvert S\rvert = 108$	$\{f, g, n\}$: $\mathbf{G}\neg n$, $\neg n\,\mathbf{U}\,f$, $\mathbf{F}(f \wedge \mathbf{F}g)$, $\mathbf{F}f \wedge \mathbf{G}\neg n$, $\neg n\,\mathbf{U}\,(f \wedge \mathbf{F}g)$; $\{e, g, n\}$: $\mathbf{F}(e \wedge \mathbf{F}g)$; $\{a, b, c\}$: $\mathbf{F}(a \wedge \mathbf{F}b)$
Taxi, $\lvert S\rvert = 101$	$\{p, g\}$: $\mathbf{F}g$, $\mathbf{F}(p \wedge \mathbf{F}g)$, $\mathbf{G}(p \to \mathbf{F}g)$
DoorKey 5×5, $\lvert S\rvert = 62$	$\{k, o, g\}$: $\mathbf{F}g$, $\neg o\,\mathbf{U}\,k$, $\mathbf{F}(o \wedge \mathbf{F}g)$, $\mathbf{G}(k \to \mathbf{F}o)$
DoorKey 6×6, $\lvert S\rvert = 154$	$\{k, o, g\}$: $\mathbf{F}g$, $\mathbf{F}(k \wedge \mathbf{F}g)$

RQ1. **(Identification).** Does the learner return a hypothesis equivalent to the target, and how do the stopping rule and the order of rivals affect this?

RQ2. **(Cost).** How many membership queries and how much time does the learner need, how do they grow with the size of the target, and how much do the order of rivals and the implementation change them?

RQ3. **(Natural-language hints).** How do correct hints affect the number of queries, candidate generation and the running time, and how do incorrect hints affect identification?

A. Experimental setup

a) Benchmarks.: We use two benchmark sets (Table II). The *free* set contains 20 LTLf targets over $\mathsf{AP} = \{a, b, c\}$, so $|\Sigma| = 8$, with sizes 2 to 7 and temporal depth at most 3; here queries may be arbitrary traces. The *environment* set contains 16 targets in four standard benchmarks: the Office world from the reward-machine literature [33], the deterministic version of Taxi [12] from Gymnasium [34], and the DoorKey 5×5 and 6×6 layouts of MiniGrid [8]. Each benchmark is represented as a finite deterministic labelled transition system with a fixed start state; states at which the original episode ends are made absorbing. Queries are restricted to feasible traces. Appendix A describes the environments and their adaptations. Each target uses the propositions listed in Table II.

In the Office world, f, e, g and n mark coffee, mail, the office and plants, and a, b, c mark room corners; the targets are parts of the four Office tasks, for example $\mathbf{F}(f \wedge \mathbf{F}g)$ (bring coffee to the office). In Taxi, p holds while the passenger is in the taxi and g once the passenger is delivered. In DoorKey, k holds while the agent carries the key, o once the door is open, and g on the goal.

b) Learner variants.: Every run uses the enumeration-based version space with the class schedule by size (Section IV-A) and the progression-based DISTINGUISH after the short-trace pool T_ℓ (Section IV-C). For a target φ_T we search $\mathcal{L}_{n,d}$ with $n = |\varphi_T| + 1$ and $d = 2$, or $d = 3$ for the two free targets of temporal depth 3; sizes count syntax-tree nodes, with $\neg p$ counted as one node. The bound n is thus assumed to be known up to one, a limitation we return to in Section VIII. INITIALIZE labels $L = 10$ traces: uniformly random traces of length 1 to 6 for free targets, and rollouts with uniformly random actions and 1 to 10 labels for environments. The first hypothesis is a smallest consistent candidate; when a witness refutes the hypothesis, the compared rival replaces it. The comparison budget is 100 (under the exhaustive rule, only comparisons that lead to a query count) and was never exhausted.

A variant of Algorithm 1 is fixed by its stopping rule and its order of rivals (Section IV-B):

- *Stopping rule. Top-*3 is the top-K rule with $K = 3$: a round compares φ_H with the three highest-ranked rivals, and refinement ends after a round without a query. *Exhaustive* is $K = \infty$: a round compares φ_H with the rivals in order, removes those equivalent to φ_H (relative to $\mathcal{E}$ in environments) without a query, and refinement ends only when no remaining candidate is distinguishable from φ_H. Only the exhaustive rule is covered by the correctness argument of Section IV.
- *Order of rivals. Enumeration* (smallest first); *semantic*, closest first under d_{sem} on $r = 200$ probe traces (200 rollouts in environments), and its reverse *semantic-far*; and *split*.

RQ1 and RQ2 report these four orders under both rules without hints (1,440 runs). We also ran four further variants (random order, closest first under a syntactic and a combined distance, and a query that splits the version space directly; 1,440 runs), which we omit for space. RQ3 runs enumeration order under both rules with eight hint settings, one of them without hints (2,660 runs). For RQ2, we also compare two alternative implementations on the free targets under the exhaustive rule: the automata-based DISTINGUISH with MONA, and the SAT-based version space.

c) Teacher and metrics.: A simulated teacher answers membership queries by evaluating φ_T. A run is *successful* (Succ.) if the final hypothesis satisfies $\varphi_H \equiv \varphi_T$ (free targets) or $\varphi_H \equiv_{\mathcal{E}} \varphi_T$ (environments); both are decided exactly by a complete product search that is used only for evaluation. A run is *exact* if $\varphi_H = \varphi_T$ up to notation. MQ counts all labelled traces, including the $L = 10$ initial ones. Eq. counts the comparisons under the exhaustive rule that find a rival equivalent to φ_H; they remove the rival without a query. Cand. counts the formulas generated by the enumerator, and Time is the wall-clock time of a run.

d) Runs.: Every configuration is run with seeds 41 to 45, giving 100 runs on the free set and 80 runs on the environment set. All numbers in this section except Table V come from one execution of all 5,540 runs; Section VI gives the hardware.

B. RQ1: Identification

a) Exhaustive rule.: Table III shows that under the exhaustive rule every run succeeds, for every order of rivals: 100 of 100 free and 80 of 80 environment runs. This is the outcome predicted by the correctness argument of Section IV, since every target lies in the searched class $\mathcal{L}_{n,d}$. In environments, the 30-action horizon of DISTINGUISH (Section VI) could leave a distinguishable rival in the version space; it did not affect any final hypothesis.

*b) Top-*3 *rule.:* Under the top-3 rule, success depends on the order of rivals: from 64 (semantic) to 100 (semantic-far, split) of 100 free runs, and from 40 to 64 of 80 environment runs. On free traces, all failures concern targets of size 5 to 7. Since the comparison budget was never exhausted, every failed run ended after a round in which the three shortlisted rivals yielded no query, although distinguishable candidates remained. Closest-first orders fail most often because the closest rivals of a hypothesis are often equivalent to it, for instance rewritings with $d_{\text{sem}} = 0$ on the probe set. Orders that favour behaviourally different rivals (semantic-far, split) avoid this on free traces. In environments the effect is stronger, because many candidates are equivalent on feasible traces: in the Office world, for example, every formula that requires a labelled cell at the first position is equivalent to false. With enumeration order, only 5 of the 35 Office runs succeed under the top-3 rule.

c) Exact identification.: Under the exhaustive rule, every order returns the target formula itself in 85 of 100 free runs; the other runs return an equivalent formula. In environments, 30 of 80 runs return the target formula, since many formulas are equivalent on feasible traces, e.g. $\mathbf{F}(g)$ and $\mathbf{F}(p \wedge \mathbf{F}g)$ in Taxi, where delivery requires a pickup.

Answer to RQ1. Under the exhaustive rule, the learner identifies every benchmark target, up to $\equiv$ or $\equiv_{\mathcal{E}}$, for every order of rivals. Under the top-3 rule it succeeds in 64 to 100 of 100 free and 40 to 64 of 80 environment runs; closest-first orders fail most often, because their shortlists consist of rivals equivalent to the hypothesis.

C. RQ2: Cost

a) Queries.: With the exhaustive rule and enumeration order, a run asks 16.2 queries on average on free targets (standard deviation 4.5, range 10 to 32) and 12.5 in environments (standard deviation 3.8, range 5 to 21), that is, 6.2 and 2.5 queries beyond the initial sample. Table IV shows that the number of queries grows with the size of the target, from 10.6 for size 2 to 20.9 for size 7 on free targets.

b) Order of rivals and stopping rule.: Under the exhaustive rule, the order of rivals changes the mean number of queries by at most 3% on free targets (15.7 for split to 16.2 for enumeration order) and 2% in environments (12.2 to 12.5; Table III). Compared with the top-3 rule under the same order, the exhaustive rule asks 0 to 2.5 more queries on free targets and 1.8 to 3.8 more in environments. These are the queries that the top-3 rule omits; for instance, the low mean of semantic

TABLE III

RQ1 AND RQ2: IDENTIFICATION AND COST BY ORDER OF RIVALS AND STOPPING RULE. ALL RUNS USE THE ENUMERATION-BASED VERSION SPACE, THE PROGRESSION-BASED DISTINGUISH AND NO HINTS. TOP-3: TOP-K RULE WITH $K = 3$; EXHAUSTIVE: $K = \infty$. SUCC.: RUNS WITH $\varphi_H \equiv \varphi_T$ (FREE, OUT OF 100) OR $\varphi_H \equiv_{\mathcal{E}} \varphi_T$ (ENVIRONMENTS, OUT OF 80). EXACT: RUNS WITH $\varphi_H = \varphi_T$ UP TO NOTATION; UNDER THE EXHAUSTIVE RULE, 85 FREE AND 30 ENVIRONMENT RUNS FOR EVERY ORDER. MQ: MEAN MEMBERSHIP QUERIES PER RUN, INCLUDING THE 10 INITIAL TRACES; TOP-3 MEANS INCLUDE FAILED RUNS. EQ.: MEAN COMPARISONS PER RUN UNDER THE EXHAUSTIVE RULE THAT FOUND A RIVAL EQUIVALENT TO φ_H.

	Free traces						**Environments**					
	Top-3			Exhaustive			Top-3			Exhaustive		
Order of rivals	Succ.	Exact	MQ	Succ.	MQ	Eq.	Succ.	Exact	MQ	Succ.	MQ	Eq.
Enumeration	92	77	15.5	100	16.2	14.2	40	20	8.7	80	12.5	310
Semantic (closest)	64	56	13.5	100	15.9	20.1	40	20	8.6	80	12.2	310
Semantic (farthest)	100	85	16.1	100	16.1	11.9	41	20	9.5	80	12.5	308
Split	100	85	15.7	100	15.7	11.8	64	25	10.6	80	12.4	296

TABLE IV

RQ2: COST BY TARGET SIZE UNDER THE EXHAUSTIVE RULE WITH ENUMERATION ORDER (NO HINTS). N: RUNS; MQ: MEAN (MINIMUM–MAXIMUM); TIME: MEAN SECONDS PER RUN; MAX: MAXIMUM SECONDS PER RUN; CAND.: MEAN NUMBER OF GENERATED FORMULAS, IN THOUSANDS.

	Free traces					**Environments**		
$\lvert\varphi_T\rvert$	N	MQ	Time	Max	Cand.	N	MQ	Time
2	10	10.6 (10–13)	0.01	0.03	0.2	20	8.7 (5–11)	0.02
3	15	11.4 (10–13)	0.01	0.02	1.0	10	10.5 (7–13)	0.02
5	35	15.7 (10–20)	0.10	0.11	55.8	45	14.0 (9–21)	0.19
6	20	18.6 (14–22)	1.07	1.32	536	5	18.4 (17–19)	1.02
7	20	20.9 (13–32)	7.94	10.70	3,944	–	–	–

TABLE V

RQ2: THREE IMPLEMENTATIONS ON THE FREE TARGETS (EXHAUSTIVE RULE, ENUMERATION ORDER, NO HINTS). PROGR.: ENUMERATED VERSION SPACE AND PROGRESSION-BASED DISTINGUISH, AS IN ALL OTHER RUNS; MONA: THE SAME WITH AUTOMATA-BASED DISTINGUISH; SAT: SAT-BASED VERSION SPACE WITH PROGRESSION-BASED DISTINGUISH. TIME: MEAN SECONDS PER RUN; MEM.: MEAN PEAK MEMORY IN MB. EACH RUN HAS ITS OWN PROCESS, WHICH ADDS ABOUT 0.7 S, SO TIMES ARE NOT COMPARABLE WITH TABLE IV. ROWS FOR $n = 9, 10$: THE FOUR SIZE-7 TARGETS WITH SEEDS 41 AND 42 AND A LIMIT OF 600 S (ONE RUN EACH FOR PROGR. AND MONA); THE SAT ENTRY GIVES THE RUNS THAT FINISHED.

		Progr.		**MONA**		**SAT**	
$\lvert\varphi_T\rvert$	n	Time	Mem.	Time	Mem.	Time	Mem.
2	3	0.83	281	0.78	281	0.89	315
3	4	0.79	281	0.80	281	0.98	316
5	6	0.87	290	0.97	290	1.63	356
6	7	1.81	350	1.98	352	3.03	415
7	8	8.82	702	9.92	728	22.35	486
7	9	70.2	3,471	69.6	3,858	2/8	579
7	10	–	–	–	–	0/8	709
Exact		85		85		50	
MQ		16.2		16.2		16.1	

ordering under the top-3 rule (13.5) reflects runs that end early, 36 of them without identifying the target. Equivalence checks cost no queries: the exhaustive rule removes 11.8 to 20.1 equivalent rivals per run on free targets, most under closest-first orders, and 296 to 310 in environments.

c) Running time.: The mean running time per run hardly depends on the variant: 1.83 to 1.84 s on free targets and 0.15 to 0.18 s in environments, for every order and both rules. It grows with the size bound n, together with the number of generated formulas: for size-7 targets ($n = 8$), the enumerator generates up to 5.1 million formulas and a run takes 7.9 s on average and at most 10.7 s. We did not include targets of size 8. With $n = 9$, a single run (target $\mathbf{G}((a \wedge b) \rightarrow \mathbf{X}c)$, seed 41) generated 31.4 million formulas and took 70 s and 3.4 GB of memory (Table V).

d) Alternative implementations.: Table V compares the default implementation with the automata-based DISTINGUISH and the SAT-based version space. All three learn every free target, with 16.1 to 16.2 queries on average. With MONA, the hypotheses and queries are identical in all 100 runs, and runs take at most 13% longer: most comparisons end in the short-trace pool, and only the others build automata. The SAT-based version space is slower at every size, 22.4 s against 8.8 s for size-7 targets. With $n = 9$, only 2 of 8 runs finish within 600 s, and with $n = 10$ none does, although the hypothesis is already equivalent to the target in all 16 runs: the remaining time is spent blocking encodings, since a formula has one encoding per order of its nodes and the exhaustive rule must exclude all of them. In return, the SAT-based construction stores no formulas and needs 0.6 GB with $n = 9$, against 3.4 GB. It also returns the target formula itself less often (50 against 85 of 100 runs), since it does not apply the enumerator's simplifications.

Answer to RQ2. With the exhaustive rule, the learner needs 16.2 queries per free target and 12.5 per environment target on average, 10 of which are the initial sample; the number grows from about 11 for size-2 targets to 21 for size-7 targets. The order of rivals changes it by at most 3%. The running time grows with the number of generated candidates and hence with the size bound: with $n = 9$, one run already takes about a minute and 3.4 GB of memory. Neither alternative implementation lowers the cost: automata-based comparisons give the same results slightly more slowly, and the SAT-based version space saves memory but does not finish the exhaustive search for $n \geq 9$ within 600 s.

TABLE VI

RQ3: EFFECT OF HINTS. ALL RUNS USE ENUMERATION ORDER OF RIVALS AND THE ENUMERATION-BASED VERSION SPACE. SUCC.: SUCCESSFUL RUNS UNDER THE TOP-3 AND THE EXHAUSTIVE RULE. MQ, CAND. (GENERATED FORMULAS, IN THOUSANDS) AND TIME (SECONDS) ARE MEANS UNDER THE EXHAUSTIVE RULE. ROWS MARKED † COVER ONLY THE TARGETS THAT RECEIVE A LEVEL-2 HINT (70 FREE AND 55 ENVIRONMENT RUNS); NONE† IS THE RUN WITHOUT HINTS ON THESE TARGETS.

	Free traces					**Environments**				
	Succ.		Exhaustive			Succ.		Exhaustive		
Hints	Top-3	Exh.	MQ	Cand.	Time	Top-3	Exh.	MQ	Cand.	Time
None	92/100	100/100	16.2	916	1.83	40/80	80/80	12.5	57	0.18
Level 1, ranking	95/100	100/100	16.0	916	1.84	64/80	80/80	12.4	57	0.20
Level 1, pruning	94/100	100/100	14.6	710	1.56	54/80	80/80	10.5	46	0.12
None†	66/70	70/70	16.0	964	1.94	25/55	55/55	13.7	77	0.23
Level 2†	67/70	70/70	14.1	964	1.96	25/55	55/55	11.9	77	0.23
Level 1 pruning + Level 2†	67/70	70/70	14.0	964	1.96	45/55	55/55	11.9	77	0.27
Incorrect Level 1, ranking	93/100	100/100	15.7	916	1.84	45/80	80/80	12.6	57	0.20
Incorrect Level 1, pruning	67/100	74/100	13.7	369	0.76	43/80	43/80	8.8	11	0.04
Incorrect Level 2	53/100	58/100	15.1	1,290	2.63	–	–	–	–	–

D. RQ3: Natural-language hints

a) Simulated hints.: A simulated teacher supplies the hints. At Level 1, it selects the question types whose formula families $\mathcal{H}_c$ contain φ_T; 10 of the 20 free targets and 14 of the 16 environment targets receive a type, and the teacher answers *unsure* for the others. At Level 2, it states the strongest instance h of a fixed set of requirement templates such that $\varphi_T \models h$ (checked exactly), h is not valid, and $h \not\equiv \varphi_T$. For example, $\mathbf{G}(a \rightarrow \mathbf{X}b)$ receives $h = \mathbf{G}(a \rightarrow \mathbf{F}b)$. Such a hint exists for 14 free and 11 environment targets. An *incorrect* Level-1 hint names a question type whose families do not contain φ_T; an incorrect Level-2 hint is the instance h with $\varphi_T \not\models h$ for which the fewest traces of T_ℓ satisfy φ_T but violate h. Every target receives both incorrect hints. We use Level-1 hints for ranking or pruning and Level-2 hints for pruning (Section IV-D), with $\ell = 4$ for free targets, and enumeration order throughout. We report incorrect Level-2 hints only for free targets: in environments, each of them is implied by φ_T on all feasible traces or violated only by traces of 14 to 26 labels.

b) Correct hints.: Used for ranking only, Level-1 hints leave the number of queries essentially unchanged (16.0 against 16.2 on free targets, 12.4 against 12.5 in environments; Table VI). They raise the success of the top-3 rule in environments from 40 to 64 of 80 runs, because the hinted families move distinguishable rivals into the shortlist. Used for pruning, Level-1 hints reduce the queries by 10% on free targets (16.2 to 14.6) and by 16% in environments (12.5 to 10.5), the generated candidates by 22% and 20%, and the running time on free targets by 15%. Level-2 hints reduce the queries on the hinted targets by 12% on free targets (16.0 to 14.1) and by 14% in environments (13.7 to 11.9). They do not reduce candidate generation, because the requirement is checked after the candidates are generated. Combining both levels gives no further reduction in queries, but raises the success of the top-3 rule in environments from 25 to 45 of 55 runs. With correct hints, the exhaustive rule succeeds in every run.

c) Incorrect hints.: Incorrect hints used for ranking do not affect success under the exhaustive rule (100 of 100 and 80 of 80 runs), in line with Lemma 2, and change the mean number of queries by at most 3%. Incorrect hints that exclude candidates do affect it: with incorrect Level-1 pruning, 74 of 100 free and 43 of 80 environment runs succeed under the exhaustive rule, and with an incorrect Level-2 hint, 58 of 100. The search widens only when a level contains no consistent candidate. If the hinted level contains a formula consistent with the sample, the search never widens and the learner returns an incorrect hypothesis. For example, for $\varphi_T = \mathbf{GF}a$ and $h = \mathbf{F}a$, which φ_T violates only on the empty trace, the learner returned $\mathbf{FG}a$, which differs from φ_T only on ε.

Answer to RQ3. Correct pruning hints reduce the queries by 10 to 16% and the generated candidates by about a fifth; ranking hints raise the success of the top-3 rule and are robust to errors. Incorrect pruning hints lower the success rate to 54 to 74%, so they need a check, such as asking the teacher to confirm the hinted requirement.

VIII. CONCLUSION

We presented an active-learning framework for LTLf in which the teacher only labels traces and answers natural-language questions. The learner identified every benchmark target with few queries, and hints reduced the queries further. Future work includes a study with human users and noisy labels, translating hints with large language models, and scaling to larger formulas.

REFERENCES

[1] Angluin, D.: Learning regular sets from queries and counterexamples. Information and Computation **75**(2), 87–106 (1987). doi:10.1016/0890-5401(87)90052-6

[2] Angluin, D.: Queries and concept learning. Machine Learning **2**(4), 319–342 (1988). doi:10.1023/A:1022821128753

[3] Bacchus, F., Kabanza, F.: Using temporal logics to express search control knowledge for planning. Artificial Intelligence **116**(1–2), 123–191 (2000). doi:10.1016/S0004-3702(99)00071-5

[4] Baharisangari, N., Ruan, Y., Zhao, C., Xu, Z.: A logic-based approach to causal discovery: Signal temporal logic perspective. In: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI) (2025), https://www.ijcai.org/proceedings/2025/529

[5] Bauer, A., Leucker, M., Schallhart, C.: Runtime verification for LTL and TLTL. ACM Transactions on Software Engineering and Methodology **20**(4), 14:1–14:64 (2011). doi:10.1145/2000799.2000800

[6] Bombara, G., Belta, C.: Offline and online learning of signal temporal logic formulae using decision trees. ACM Transactions on Cyber-Physical Systems **5**(3) (2021). doi:10.1145/3433994

[7] Camacho, A., McIlraith, S.A.: Learning interpretable models expressed in linear temporal logic. In: Proceedings of the International Conference on Automated Planning and Scheduling. vol. 29, pp. 621–630 (2019). doi:10.1609/icaps.v29i1.3529

[8] Chevalier-Boisvert, M., Dai, B., Towers, M., Perez-Vicente, R., Willems, L., Lahlou, S., Pal, S., Castro, P.S., Terry, J.: Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In: Advances in Neural Information Processing Systems. vol. 36 (2023). doi:10.52202/075280-3209, https://proceedings.neurips.cc/paper_files/paper/2023/hash/e8916198466e8ef218a2185a491b49fa-Abstract.html

[9] Chiariello, F.: Learning temporal properties from event logs via sequential analysis. In: 31st International Symposium on Temporal Representation and Reasoning (TIME 2024). Leibniz International Proceedings in Informatics, vol. 318, pp. 14:1–14:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2024). doi:10.4230/LIPIcs.TIME.2024.14

[10] Clopper, C.J., Pearson, E.S.: The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika **26**(4), 404–413 (1934). doi:10.1093/biomet/26.4.404

[11] De Giacomo, G., Vardi, M.Y.: Linear temporal logic and linear dynamic logic on finite traces. In: Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI). pp. 854–860 (2013), https://www.cs.rice.edu/~vardi/papers/ijcai13.pdf

[12] Dietterich, T.G.: Hierarchical reinforcement learning with the MAXQ value function decomposition. Journal of Artificial Intelligence Research **13**, 227–303 (2000). doi:10.1613/jair.639

[13] Dwyer, M.B., Avrunin, G.S., Corbett, J.C.: Patterns in property specifications for finite-state verification. In: Proceedings of the 21st International Conference on Software Engineering (ICSE). pp. 411–420. ACM (1999). doi:10.1145/302405.302672

[14] Elgaard, J., Klarlund, N., Møller, A.: MONA 1.x: New techniques for WS1S and WS2S. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 1427, pp. 516–520. Springer (1998). doi:10.1007/BFb0028773

[15] Fuggitti, F.: LTLf2DFA. Software, Zenodo (2019). doi:10.5281/zenodo.3888410, https://github.com/whitemech/LTLf2DFA

[16] Gavran, I., Darulova, E., Majumdar, R.: Interactive synthesis of temporal specifications from examples and natural language. Proceedings of the ACM on Programming Languages **4**(OOPSLA), 201:1–201:26 (2020). doi:10.1145/3428269

[17] Hasanbeig, H., Kroening, D., Abate, A.: Certified reinforcement learning with logic guidance. Artificial Intelligence **322**, 103949 (2023). doi:10.1016/j.artint.2023.103949

[18] Kamath, A., Zhang, S., Xu, C., Ugare, S., Singh, G., Misailovic, S.: Enforcing temporal constraints for LLM agents. arXiv:2512.23738 (2025)

[19] Kress-Gazit, H., Eder, K., Hoffman, G., Admoni, H., Argall, B., Ehlers, R., Heckman, C., Jansen, N., Knepper, R.A., Křetínský, J., Levy-Tzedek, S., Li, J., Murphey, T.D., Riek, L.D., Sadigh, D.: Formalizing and guaranteeing human-robot interaction. Communications of the ACM **64**(9), 78–84 (2021). doi:10.1145/3433637

[20] Li, D., Cai, M., Vasile, C.I., Tron, R.: Learning signal temporal logic through neural network for interpretable classification. In: American Control Conference (ACC). pp. 1907–1914. IEEE (2023), https://arxiv.org/abs/2210.01910

[21] Liang, K., Cardona, G.A., Kamale, D., Vasile, C.I.: Learning optimal signal temporal logic decision trees for classification: A max-flow MILP formulation. In: IEEE Conference on Decision and Control (CDC). IEEE (2024), https://arxiv.org/abs/2407.21090

[22] Linard, A., Tůmová, J.: Active learning of signal temporal logic specifications. In: IEEE 16th International Conference on Automation Science and Engineering (CASE). pp. 779–785. IEEE (2020). doi:10.1109/CASE48305.2020.9216778

[23] Liu, J.X., Yang, Z., Idrees, I., Liang, S., Schornstein, B., Tellex, S., Shah, A.: Grounding complex natural language commands for temporal tasks in unseen environments. In: Proceedings of the 7th Conference on Robot Learning. Proceedings of Machine Learning Research, vol. 229, pp. 1084–1110. PMLR (2023), https://proceedings.mlr.press/v229/liu23d.html

[24] Luo, W., Liang, P., Du, J., Wan, H., Peng, B., Zhang, D.: Bridging LTLf inference to GNN inference for learning LTLf formulae. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 36, pp. 9849–9857 (2022). doi:10.1609/aaai.v36i9.21221

[25] de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems – TACAS 2008. Lecture Notes in Computer Science, vol. 4963, pp. 337–340. Springer (2008). doi:10.1007/978-3-540-78800-3_24

[26] Neider, D., Gavran, I.: Learning linear temporal properties. In: Formal Methods in Computer Aided Design (FMCAD). pp. 1–10. IEEE (2018). doi:10.23919/FMCAD.2018.8603016, https://arxiv.org/abs/1806.03953

[27] Pnueli, A.: The temporal logic of programs. In: 18th Annual Symposium on Foundations of Computer Science (FOCS). pp. 46–57. IEEE (1977). doi:10.1109/SFCS.1977.32

[28] Rabin, M.O., Scott, D.S.: Finite automata and their decision problems. IBM Journal of Research and Development **3**(2), 114–125 (1959). doi:10.1147/rd.32.0114

[29] Raha, R., Roy, R., Fijalkow, N., Neider, D.: Scalable anytime algorithms for learning fragments of linear temporal logic. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Springer (2022). doi:10.1007/978-3-030-99524-9_14

[30] Roy, R., Gaglione, J.R., Baharisangari, N., Neider, D., Xu, Z., Topcu, U.: Learning interpretable temporal properties from positive examples only. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 37, pp. 6507–6515 (2023). doi:10.1609/aaai.v37i5.25800

[31] Roy, R., Pote, Y., Parker, D., Kwiatkowska, M.: Learning probabilistic temporal logic specifications for stochastic systems. In: IJCAI. pp. 4642–4650. ijcai.org (2025)

[32] Seshia, S.A., Sadigh, D., Sastry, S.S.: Toward verified artificial intelligence. Communications of the ACM **65**(7), 46–55 (2022)

[33] Toro Icarte, R., Klassen, T.Q., Valenzano, R., McIlraith, S.A.: Reward machines: Exploiting reward function structure in reinforcement learning. Journal of Artificial Intelligence Research **73**, 173–208 (2022). doi:10.1613/jair.1.12440

[34] Towers, M., Kwiatkowski, A., Balis, J., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Terry, J., Pierré, A., Schulhoff, S., Tai, J.J., Tan, H., Younis, O.G.: Gymnasium: A standard interface for reinforcement learning environments. In: Advances in Neural Information Processing Systems. vol. 38 (2025). doi:10.52202/085713-4916, https://proceedings.neurips.cc/paper_files/paper/2025/hash/d7ff1795e8527f6443371c3933bdb52b-Abstract-Datasets_and_Benchmarks_Track.html

[35] Vaandrager, F.: Model learning. Communications of the ACM **60**(2), 86–95 (2017). doi:10.1145/2967606

[36] Valiant, L.G.: A theory of the learnable. Communications of the ACM **27**(11), 1134–1142 (1984). doi:10.1145/1968.1972

[37] Vardi, M.Y., Wolper, P.: An automata-theoretic approach to automatic program verification. In: Proceedings of the First Symposium on Logic in Computer Science (LICS). pp. 332–344. IEEE Computer Society (1986), https://www.cs.rice.edu/~vardi/papers/lics86.pdf

[38] Yang, Z., Raman, S.S., Shah, A., Tellex, S.: Plug in the safety chip: Enforcing constraints for LLM-driven robot agents. In: IEEE International Conference on Robotics and Automation (ICRA). pp. 14435–14442. IEEE (2024). doi:10.1109/ICRA57147.2024.10611447

APPENDIX A
BENCHMARK ENVIRONMENTS

The environment experiments use four configurations from three benchmark families: Office world, Taxi, and MiniGrid DoorKey. Table VII lists their reachable state counts and the number of target formulas. The target formulas are given in Table II.

TABLE VII
ENVIRONMENT CONFIGURATIONS USED IN THE EXPERIMENTS.

Environment	States	Targets
Office world	108	7
Taxi	101	3
DoorKey 5×5	62	4
DoorKey 6×6	154	2

a) Common representation.: Each environment is represented as a finite deterministic labelled transition system with a fixed start state. A trace records the label of the initial state followed by the label after each action. Labels are projected onto the propositions used by the target formula. Random rollouts choose actions uniformly. Successful delivery in Taxi and goal states in DoorKey are made absorbing: subsequent actions leave the state and its label unchanged.

b) Office world.: The Office world [33] is a 12×9 grid divided into 3×3 rooms connected by doorways. The agent starts at $(2, 1)$ and moves up, right, down, or left; a move blocked by a wall leaves it in place. The propositions mark coffee (f), mail (e), the office (g), plants (n), and four designated locations (a, b, c, d). The seven targets use the proposition sets $\{f, g, n\}$, $\{e, g, n\}$, or $\{a, b, c\}$ and specify avoidance or ordered visits to labelled locations.

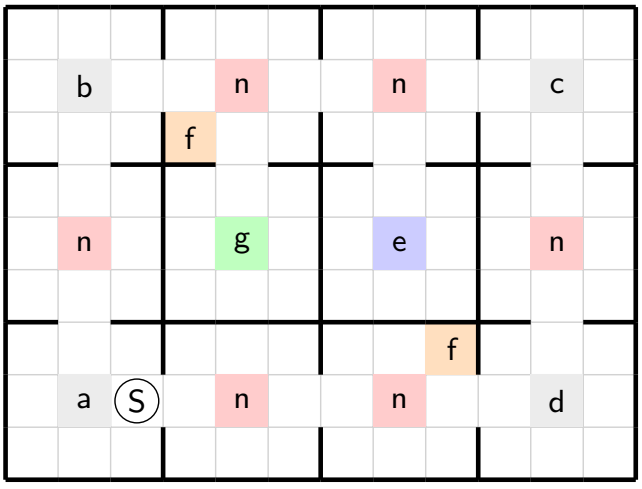


Fig. 1. Office world, drawn from the implemented map. Thick lines denote walls and gaps denote doorways. S marks the start; the other labels identify coffee (f), mail (e), the office (g), plants (n), and locations a, b, c, d.

c) Taxi.: The Taxi benchmark [12] uses Gymnasium's `Taxi-v4` [34], with rain and passenger relocation disabled to obtain deterministic transitions. Reset seed 0 fixes the taxi's initial position, passenger location, and destination. The six actions are south, north, east, west, pickup, and dropoff. Proposition p holds while the passenger is in the taxi, and g holds once the passenger has been delivered. The three targets specify delivery, pickup followed by delivery, or a response requirement for carrying a passenger.

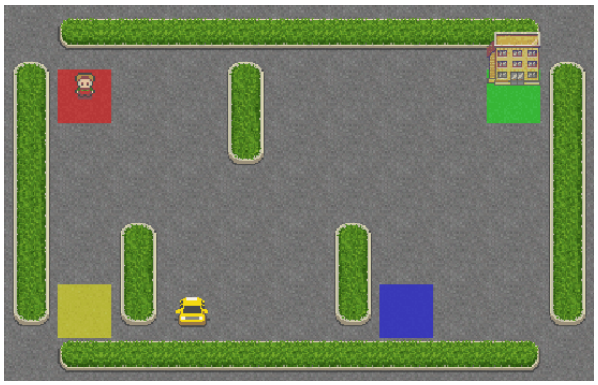
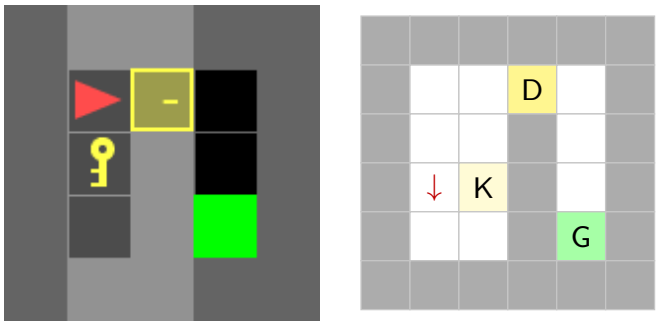


Fig. 2. Illustrative Taxi environment. Image: still frame from the official Gymnasium documentation. The pictured taxi and passenger positions need not match the experiment's reset seed.

d) MiniGrid DoorKey.: The DoorKey benchmarks [8] use `MiniGrid-DoorKey-5x5-v0` and `MiniGrid-DoorKey-6x6-v0`, both with reset seed 0. States record the agent's position and orientation, the carried object, and the grid configuration. The available actions are turn left, turn right, move forward, pickup, and toggle; drop and done are omitted, and the original episode step limit is ignored. Proposition k denotes carrying the key, o an open door, and g reaching the goal. The 5×5 layout has four targets and the 6×6 layout has two, covering goal reachability and requirements involving the key and door.



(a) DoorKey 5×5 (b) DoorKey 6×6

Fig. 3. DoorKey environments. (a) Illustrative 5×5 layout from the official MiniGrid documentation; its initial configuration may differ from the experiment's. (b) Grid drawn from the 6×6 experiment with reset seed 0: K denotes the key, D the locked door, G the goal, and the arrow the agent's initial position and orientation. In (a), the key and door are yellow, the goal is green, and the agent is a red triangle.

APPENDIX B
DISTINGUISHING BY FORMULA PROGRESSION

The automata-based procedure constructs an automaton for every compared formula. Our implementation instead explores the symmetric-difference automaton on the fly. Its states are *progressed* formulas [3]: after a letter is read, a formula is rewritten into the requirement on the rest of the word. We describe this procedure and show that it decides equivalence over finite words.

a) Empty words and negation normal form.: Progression uses the endpoint semantics from the preliminaries: $\varepsilon \not\models p$, $\varepsilon \models \neg p$, $\varepsilon \not\models \mathbf{X}\varphi$, and $\varepsilon \not\models \varphi\,\mathbf{U}\,\psi$. We use the weak next operator $\widetilde{\mathbf{X}}$, with $w, i \models \widetilde{\mathbf{X}}\varphi$ iff $i+1 \geq |w|$ or $w, i+1 \models \varphi$, and release $\varphi\,\mathbf{R}\,\psi = \neg(\neg\varphi\,\mathbf{U}\,\neg\psi)$. Since $\neg\mathbf{X}\varphi \equiv \widetilde{\mathbf{X}}\neg\varphi$ and $\neg(\varphi\,\mathbf{U}\,\psi) \equiv \neg\varphi\,\mathbf{R}\,\neg\psi$ over Σ^*, every formula has an equivalent *negation normal form* (NNF), in which negation occurs only in front of propositions. A formula is *basic* if it is a literal (p or $\neg p$) or its root operator is $\mathbf{X}$, $\widetilde{\mathbf{X}}$, $\mathbf{U}$ or $\mathbf{R}$. Every NNF formula is a combination of basic formulas, true and false by $\wedge$ and $\vee$. We write $\mathbf{F}\mathrm{true}$ for $\mathrm{true}\,\mathbf{U}\,\mathrm{true}$, which holds exactly on nonempty words, and $\mathbf{G}\mathrm{false}$ for $\mathrm{false}\,\mathbf{R}\,\mathrm{false}$, which holds exactly on ε.

Definition 4 (Progression)**.** *For an NNF formula φ and a letter $a \in \Sigma$, the formula* $\mathrm{prog}(\varphi, a)$ *is defined by* $\mathrm{prog}(\mathrm{true}, a) = \mathrm{true}$, $\mathrm{prog}(\mathrm{false}, a) = \mathrm{false}$ *and*

$$
\begin{aligned}
\mathrm{prog}(p, a) &= \mathrm{true} \text{ if } p \in a, \text{ and } \mathrm{false} \text{ otherwise},\\
\mathrm{prog}(\neg p, a) &= \mathrm{false} \text{ if } p \in a, \text{ and } \mathrm{true} \text{ otherwise},\\
\mathrm{prog}(\varphi \wedge \psi, a) &= \mathrm{prog}(\varphi, a) \wedge \mathrm{prog}(\psi, a),\\
\mathrm{prog}(\varphi \vee \psi, a) &= \mathrm{prog}(\varphi, a) \vee \mathrm{prog}(\psi, a),\\
\mathrm{prog}(\mathbf{X}\varphi, a) &= \varphi \wedge \mathbf{F}\mathrm{true},\\
\mathrm{prog}(\widetilde{\mathbf{X}}\varphi, a) &= \varphi \vee \mathbf{G}\mathrm{false},\\
\mathrm{prog}(\varphi\,\mathbf{U}\,\psi, a) &= \mathrm{prog}(\psi, a)\\
&\quad \vee \big(\mathrm{prog}(\varphi, a) \wedge \varphi\,\mathbf{U}\,\psi\big),\\
\mathrm{prog}(\varphi\,\mathbf{R}\,\psi, a) &= \mathrm{prog}(\psi, a)\\
&\quad \wedge \big(\mathrm{prog}(\varphi, a) \vee \varphi\,\mathbf{R}\,\psi\big).
\end{aligned}
$$

For words, $\mathrm{prog}(\varphi, \varepsilon) = \varphi$ *and* $\mathrm{prog}(\varphi, ua) = \mathrm{prog}(\mathrm{prog}(\varphi, u), a)$. *The* end value $\mathrm{fin}(\varphi) \in \{0, 1\}$ *is* 1 *for* true, $\neg p$, $\widetilde{\mathbf{X}}\varphi$ *and* $\varphi\,\mathbf{R}\,\psi$*; it is* 0 *for* false, p, $\mathbf{X}\varphi$ *and* $\varphi\,\mathbf{U}\,\psi$*; and it is the minimum (maximum) of the operands' values for* $\wedge$ *(*$\vee$*).*

Lemma 3 (Progression)**.** *For every NNF formula φ, letter $a \in \Sigma$ and word $w \in \Sigma^*$: (i) $\varepsilon \models \varphi$ iff $\mathrm{fin}(\varphi) = 1$; and (ii) $aw \models \varphi$ iff $w \models \mathrm{prog}(\varphi, a)$. Consequently, $uw \models \varphi$ iff $w \models \mathrm{prog}(\varphi, u)$ for all $u, w \in \Sigma^*$.*

Proof. Both claims are proved by induction on φ. For (i), evaluate the semantic clauses at position $0 = |\varepsilon|$: propositions, $\mathbf{X}$ and $\mathbf{U}$ require a position below $|\varepsilon|$ and are false, and their duals are true. For (ii), we use that $aw, 1 \models \chi$ iff $w, 0 \models \chi$ for every χ, since both sides evaluate χ on the same suffix; this includes $w = \varepsilon$. Literals and Boolean connectives are immediate. For $\mathbf{X}$: $aw \models \mathbf{X}\varphi$ iff $1 < |aw|$ and $aw, 1 \models \varphi$, iff $w \neq \varepsilon$ and $w \models \varphi$, iff $w \models \varphi \wedge \mathbf{F}\mathrm{true}$. For $\mathbf{U}$: $aw \models \varphi\,\mathbf{U}\,\psi$ iff $aw \models \psi$ (a witness position $j = 0$), or $aw \models \varphi$ and $aw, 1 \models \varphi\,\mathbf{U}\,\psi$ (a witness position $j \geq 1$). By the induction hypothesis for φ and ψ and the suffix property, this holds iff $w \models \mathrm{prog}(\psi, a) \vee (\mathrm{prog}(\varphi, a) \wedge \varphi\,\mathbf{U}\,\psi)$. The cases $\widetilde{\mathbf{X}}$ and $\mathbf{R}$ are dual. The statement for words follows by induction on $|u|$. □

b) Normal form.: Without simplification, progressed formulas grow with every letter. The implementation keeps each progressed formula in disjunctive normal form over basic formulas: a set of clauses, each a set of basic formulas read as a conjunction, where true is the set containing the empty clause and false is the empty set. The normalisation $\mathcal{N}$ (a) removes clauses that contain p and $\neg p$, (b) removes clauses that contain another clause (absorption), and (c) replaces a disjunction with unit clauses $\{p\}$ and $\{\neg p\}$ by true. Rules (a) and (c) are valid over Σ^* because exactly one of p and $\neg p$ holds at every position, including the end of the word; rule (b) is valid in propositional logic. Hence $\mathcal{N}(\chi) \equiv \chi$ over Σ^*. We write $\mathrm{prog}_{\mathcal{N}}(\chi, a) = \mathcal{N}(\mathrm{prog}(\chi, a))$, extend it to words as above, and call two normalised formulas equal when they are equal as sets of clauses.

Lemma 4 (Finitely many progressed formulas)**.** *Let $B(\varphi)$ be the set of basic subformulas of the NNF of φ, together with* $\mathbf{F}\mathrm{true}$ *and* $\mathbf{G}\mathrm{false}$*. For every $u \in \Sigma^*$, the formula* $\mathrm{prog}_{\mathcal{N}}(\mathcal{N}(\varphi), u)$ *is a set of subsets of $B(\varphi)$ none of which contains another. Hence at most $2^{2^{|B(\varphi)|}}$ distinct formulas are reached from φ by progression, and $|B(\varphi)| \leq 2|\varphi| + 2$.*

Proof. Call a formula *over* $B(\varphi)$ if it is a combination of elements of $B(\varphi) \cup \{\mathrm{true}, \mathrm{false}\}$ by $\wedge$ and $\vee$. We first show by induction that $\mathrm{prog}(\chi, a)$ is over $B(\varphi)$ for every subformula χ of the NNF of φ. Literals progress to true or false, and $\wedge$, $\vee$ follow from the induction hypothesis. For $\mathbf{X}\chi'$ and $\widetilde{\mathbf{X}}\chi'$, the result combines χ', which is over its own basic subformulas, with $\mathbf{F}\mathrm{true}$ or $\mathbf{G}\mathrm{false}$. For $\chi'\,\mathbf{U}\,\chi''$ and $\chi'\,\mathbf{R}\,\chi''$, the result combines the formula itself, which is in $B(\varphi)$, with $\mathrm{prog}(\chi', a)$ and $\mathrm{prog}(\chi'', a)$, which are over $B(\varphi)$ by the induction hypothesis. Moreover, $\mathbf{F}\mathrm{true}$ progresses to $\mathrm{true} \vee (\mathrm{true} \wedge \mathbf{F}\mathrm{true})$ and $\mathbf{G}\mathrm{false}$ to $\mathrm{false} \wedge (\mathrm{false} \vee \mathbf{G}\mathrm{false})$. Since progression distributes over $\wedge$ and $\vee$, induction on $|u|$ shows that every formula reached from φ is over $B(\varphi)$. The normalisation $\mathcal{N}$ turns such a formula into a set of clauses over $B(\varphi)$, and by absorption no clause contains another. There are at most $2^{2^{|B(\varphi)|}}$ such sets. Finally, each subformula of φ yields at most two subformulas of the NNF, one per polarity, so $|B(\varphi)| \leq 2|\varphi| + 2$. □

The bound counts distinct sets of clauses, not distinct languages: equivalent formulas with different normal forms only enlarge the search and do not affect its correctness.

Pairs with equal components are not expanded: by Lemma 3, no extension of their word distinguishes the two formulas.

Algorithm 2 Progression-based distinguishing procedure DISTINGUISHPROG

Require: LTLf formulas φ and ψ; a fixed order on Σ
Ensure: A shortest word $w \in \Sigma^*$ with $(w \models \varphi) \oplus (w \models \psi)$, or false if none exists

1: $s \leftarrow (\mathcal{N}(\varphi), \mathcal{N}(\psi))$
2: **if** $s_1 = s_2$ **then**
3: **return** false
4: **end if**
5: $Q \leftarrow \langle (s, \varepsilon) \rangle$; $V \leftarrow \{s\}$ ▷ FIFO queue, visited pairs
6: **while** $Q \neq \langle\rangle$ **do**
7: $((s_1, s_2), u) \leftarrow \text{DEQUEUE}(Q)$
8: **if** $\text{fin}(s_1) \neq \text{fin}(s_2)$ **then**
9: **return** u
10: **end if**
11: **for all** $a \in \Sigma$ in the fixed order **do**
12: $t \leftarrow (\text{prog}_{\mathcal{N}}(s_1, a), \text{prog}_{\mathcal{N}}(s_2, a))$
13: **if** $t_1 \neq t_2$ and $t \notin V$ **then**
14: $V \leftarrow V \cup \{t\}$; $\text{ENQUEUE}(Q, (t, ua))$
15: **end if**
16: **end for**
17: **end while**
18: **return** false

Lemma 5 (Correctness of DISTINGUISHPROG)**.** *For all LTLf formulas φ and ψ, Algorithm 2 terminates after at most $2^{2^{|B(\varphi)|}} \cdot 2^{2^{|B(\psi)|}}$ iterations of its main loop. It returns a word if and only if some $w \in \Sigma^*$ satisfies $(w \models \varphi) \oplus (w \models \psi)$, and every returned word is the first such word in length-lexicographic order. In particular, it returns* false *if and only if φ and ψ are equivalent over Σ^*.*

Proof. Invariant. If $((s_1, s_2), u)$ is enqueued, then $s_1 = \text{prog}_{\mathcal{N}}(\mathcal{N}(\varphi), u)$ and $s_2 = \text{prog}_{\mathcal{N}}(\mathcal{N}(\psi), u)$. By Lemma 3 and $\mathcal{N}(\chi) \equiv \chi$, for every $v \in \Sigma^*$, $uv \models \varphi$ iff $v \models s_1$, and $uv \models \psi$ iff $v \models s_2$.

Termination. Every pair is enqueued at most once, and by Lemma 4 there are at most $2^{2^{|B(\varphi)|}} \cdot 2^{2^{|B(\psi)|}}$ pairs.

Soundness. If u is returned, the invariant with $v = \varepsilon$ and Lemma 3(i) give $u \models \varphi$ iff $\text{fin}(s_1) = 1$ and $u \models \psi$ iff $\text{fin}(s_2) = 1$; these differ.

Completeness and minimality. Suppose that a distinguishing word exists, and let w be the first one in length-lexicographic order. For every prefix u of w, the pair reached by u has distinct components; otherwise, writing $w = uv$, the invariant gives $w \models \varphi$ iff $v \models s_1$ iff $w \models \psi$. Since words are extended by letters in the fixed order, the queue is ordered length-lexicographically, and every pair reachable from s through pairs with distinct components is enqueued with the first word, in this order, that reaches it through such pairs. Hence the pair p reached by w is enqueued with a word u' that is not after w. By the invariant, the end values of p differ, so the search returns a word no later than when p is dequeued, and that word is not after u'. By soundness the returned word distinguishes φ and ψ, so by the choice of w it equals w. If no distinguishing word exists, then by soundness no word is returned, and the check $s_1 = s_2$ in line 2 is correct because equal normal forms are equivalent. □

Lemma 5 gives the guarantee of Lemma 1 over Σ^* without constructing automata. The search builds only the pairs it reaches and stops at the first witness; its worst case is doubly exponential in $|\varphi| + |\psi|$. In the implementation, the number of pairs is bounded by a budget (Section VI); a search that exceeds it reports a failure instead of an answer.

c) Trace domain.: The initial queue contains (s, ε), so an empty trace is returned when the candidates disagree at the endpoint. This is a valid membership query under $\mathcal{W} = \Sigma^*$. Environment queries instead use the nonempty feasible traces described below.

d) Environments.: For an environment given as a finite labelled transition system, the search runs over triples of an environment state and two progressed formulas, and a triple is expanded along the transitions of the environment. By Lemma 4 there are finitely many triples, and the argument of Lemma 5 shows that the search returns a feasible distinguishing word if and only if one exists. During learning, the implementation also bounds the number of actions (Section VI); a search that reaches this bound returns no answer, which preserves soundness but not completeness.

APPENDIX C
PROOFS

This appendix proves the results of the main text whose proofs are not given there; each proof refers to the statement it proves.

Proof of Lemma 1. Automata. Both translations and the emptiness test terminate. By correctness of the LTLf-to-automaton translation [11], with the stated empty-word convention, and the Boolean product construction [28], $L(\mathcal{A}_\Delta) = \{w \in \Sigma^* \mid (w \models_{\text{LTLf}} \varphi) \oplus (w \models_{\text{LTLf}} \psi)\}$. Therefore, an accepting word is exactly a distinguishing word, and the emptiness test returns false exactly when the formulas are equivalent over finite words.

Progression. Algorithm 2 in Appendix B is the search of Section IV-C2. By Lemma 5, it terminates, returns a word if and only if a distinguishing word exists, and returns the first distinguishing word in length-lexicographic order, which is a shortest one. □

Proof of Lemma 2. The version space is

$$V(S) = \{\psi \in \mathcal{H} \mid \forall (w, y) \in S,\ (w \models \psi \iff y = 1)\},$$

where $\mathcal{H}$ is the currently searched hypothesis class. Neither the question type c nor the requirement h changes membership in $V(S)$. Each answered query, including those generated using hints, is added to S, and the version space is updated before the next hypothesis is selected. Hence every selected hypothesis satisfies all accumulated labels, irrespective of hint accuracy. □

Proof of Proposition 1. Candidate generation filters by consistency, and selection is restricted to the filtered set. The invariant is restored after every new answer. □

Proof of Theorem 1. Every label is given by φ_T, so the equivalence class of φ_T is never removed from the version space. Each query separates φ_H from a rival, so the formulas equivalent to one of them become inconsistent: every query removes at least one class, and removing an equivalent rival needs no query. Hence at most $|\mathcal{H}/{\equiv}| - 1$ refinement queries are asked. Under the exhaustive rule, refinement on $\mathcal{H}$ ends only when every remaining formula is equivalent to φ_H; since the class of φ_T remains, $\varphi_H \equiv \varphi_T$. □

Proof of Theorem 2. As in the proof of Theorem 1, with $\equiv_{\mathcal{E}}$ in place of $\equiv$: formulas in one class of $\equiv_{\mathcal{E}}$ receive the same label on every feasible query, the class of φ_T is never removed, and each feasible witness removes at least one class, so at most $N_{\mathcal{E}} - 1$ refinement queries are asked. At termination no remaining formula can be distinguished from φ_H by a feasible trace, so $\varphi_H \equiv_{\mathcal{E}} \varphi_T$. □

Proof of Theorem 3. Let $\epsilon = \ln(1/\delta)/P$. If $\mathrm{err}_{\mathcal{D}}(\varphi_H, \varphi_T) > \epsilon$, all P independent tests agree with φ_H with probability at most $(1-\epsilon)^P \leq e^{-\epsilon P} = \delta$. Hence, with probability at least $1 - \delta$, some test disagrees or the error is at most ϵ. □